* Binary Search Applications :→ Important for placements :

① Finding the square root (integral part) of an integer in logN. (logN)

sq(36) → 0 — 36 (s, e)

$m = \frac{s+e}{2} = \frac{36}{2} = 18$

$m = \frac{17}{2} = 8$

$m = \frac{7}{2} = 3$

$m = \frac{11}{2} = 5$

$\frac{12}{2} = 6$

$6 \times 6 = 36$

e = m − 1

0 — 17

e = m − 1

0 — 7

s = m + 1

4 — 7

s = m + 1

s − 7

ans = m (7)

while (s <= e)

* If answer is bigger move to left → e = m − 1
* If answer is smaller, store the ans & move to right for probable ans.

ans = m

s = m + 1;

0 — 64

e = m − 1

0 - 31

e = m − 1

0 - 14

s = m + 1

8 — 14

e = m − 1

8 — 10

e = m − 1

8 — 8

$\frac{16}{2} = 8$

$8 \times 8 = 64$

```
int sqrroot (int n)
{
ans = mid
ans = mid
}
```

35 36

3 5 6

5

25 — 5

26 — 5

37 - 6

$5 \times 5 = 25$

$3 \times 3 = 9$

$x \times x = n$

$18 \times 18 = 324$

$8 \times 8 = 64$

m = 32

$32 \times 32 = 1024$

$\frac{31}{2} = 15$

225

$\frac{14}{2} = 7$

$\frac{22}{2} = 11$

121

$\frac{18}{2} = 9$

81

* First, Last, Total Occurrences of an Element in a sorted array in logN.

m = $\frac{s+e}{2} = \frac{7}{2} = 3$ (key) 3

First Occ = 2 {1, 2, 3, 3, 3, 4, 5, 6} Last Occ = 4

0 1 2 3 4 5 6 7

ans = −1;

```
if (arr[mid] == key){
    ans = mid;
    move left for fOcc
    e = m-1;
}
else if (arr[mid] < key){
    s = m+1;
}
else {
    e = m-1;
}
```

$f_o = 2$

$l_o = 4$

$t_o = l_o - f_o + 1$

$= 4 - 2 + 1$

$= 3$

```
if (arr[mid] == key){
    ans = mid;
    move right for lsOcc
    s = m+1;
}
else if (arr[mid] < key){
    s = m+1;
}
else e = m-1;
```

Important Binary Search (logN) Applications for Coding Rounds

* Square Root Integral Part
* First, Last, total Occurrences of element in sorted array
* Search in a 2D Matrix → (LC) (UFC) (CN)
* Missing Element in a Sorted Array
* Peak in a Mountain | Peak index in an Array
* Aggressive Cows
* Book Allocation Problem
* Painter's Partition Problem

Thumbrule for Time Complexity :→

* Loops → sequential → for for for

O(n) + O(n) + O(n) ≃ O(n)

* Nested for for for

O(n) × O(n) × O(n)

$O(n^3)$

Space
* Vars → O(1)
* Array → O(n)

* is (malayalam) l r

l++ r--

false

bond

```
freq[str.charAt(i) - 'a']++
```

'b' − 'a'

98 − 97 = 1

Ascii − 97 = some index

b − a = 98 − 97 = 1

++ ++ ++ ++ ++ ++

[1 1]

0 1 2 3 4 5 6 ... 25

0